Project: Anti-Money Laundering (AML) Detection System

Prepared by:
Mazen Elsayed 8247
Youssef Attia 8164
Karim Mohamed 8237
Youssef Gaber 8115

Notebook 1 (EDA):

This Jupyter notebook, titled "01_EDA.ipynb," serves as the first step in building an Anti-Money Laundering (AML) Detection System. Exploratory Data Analysis (EDA) is a process where data scientists examine a dataset to uncover patterns, spot anomalies, and check assumptions through summaries and visualizations. Think of EDA like a detective's initial investigation: it helps understand the "crime scene" (the data) before building models to catch money launderers.

The notebook focuses on the SAML-D dataset, a large collection of financial transaction records stored in a CSV file. If the file isn't found in the specified directory, the code prompts the user to place it there. The dataset includes details like transaction times, amounts, currencies, bank locations, and whether a transaction is flagged as laundering (fraudulent). The goal is to load the data, balance it for fairness, analyze its characteristics, and apply clustering techniques to identify suspicious patterns. Clustering, in simple terms, is grouping similar data points together—like sorting fruits into baskets based on color and size—to reveal hidden structures.

The notebook uses Python libraries such as Pandas for data handling, Matplotlib and Seaborn for plotting, and Scikit-learn for advanced analysis. It processes over 9.5 million rows initially, but creates a smaller sample for efficiency.

Data Loading and Initial Inspection

The process begins by importing necessary tools and loading the full dataset from "SAML-D.csv" using a custom function from "load_data.py." This function reads the CSV file into a Pandas DataFrame, a table-like structure for organizing data.

Upon loading, the notebook displays basic information: the dataset has 9,504,852 rows and 12 columns, including "Time," "Date," "Sender_account," "Receiver_account," "Amount," "Payment_currency," "Received_currency," "Sender_bank_location," "Receiver_bank_location," "Payment_type," "Is_laundering," and "Laundering_type." The first few rows are printed to give a snapshot, showing examples like a UK pounds cash deposit of £1,459.15 marked as non-laundering.

A 500-row sample is created and saved as "sample.csv" for quick testing. This is like taking a small bite of a cake to taste it without eating the whole thing—efficient for initial checks without overwhelming computer resources.

Following loading, the notebook transitions to preparing the data for analysis, addressing a common issue in fraud detection: class imbalance.

Data Preparation and Class Balancing

Real-world fraud datasets are often imbalanced, meaning legitimate transactions vastly outnumber fraudulent ones. Here, 99.9% of transactions are legitimate (9,494,979), while only 0.1% are laundering (9,873), creating a ratio of about 962:1. This imbalance is like searching for a few counterfeit bills in a mountain of real money; models might ignore the rare fraud cases.

To fix this, the notebook drops the "Laundering_type" column, as the focus is on binary classification (laundering or not). It then balances the data by keeping all fraud cases and randomly undersampling legitimate ones to achieve a 5:1 ratio (49,365 legitimate to 9,873 fraud), resulting in 59,238 rows total. Undersampling is reducing the majority group to match the minority, like thinning a crowd to focus on suspects.

The balanced dataset is shuffled (randomly reordered) and saved as "balanced_data.csv" for use in later notebooks. This step reduces the dataset size to 0.62% of the original but makes it more suitable for modeling. The new proportions are 83.3% legitimate and 16.7% laundering, improving fairness.

With the data prepared, the notebook moves to detailed inspection using statistics and visualizations.

Dataset Information and Basic Statistics

The "info()" function reveals data types: numerical (like "Amount" as float), categorical (like "Payment_currency" as object/string), and the target "Is_laundering" as integer (0 or 1). There are no missing values, which is good—missing data can be like holes in a puzzle.

The "describe()" method provides numerical summaries: average transaction amount is about £13,909, with a wide range from £5.45 to over £12 million. Categorical stats show "UK pounds" as the most common currency (94.6% of payments) and "UK" as the dominant bank location (96% for senders).

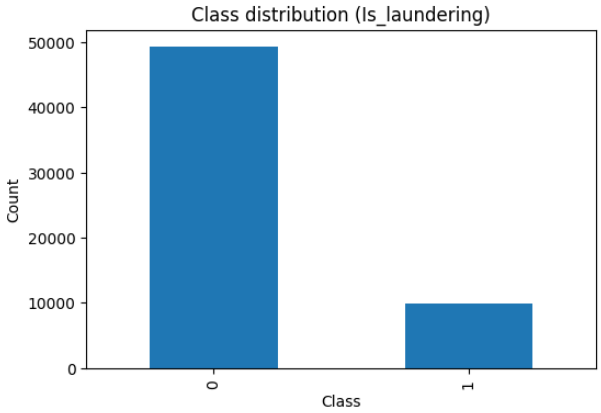
Unique values are counted: 13 currencies, 18 locations, 7 payment types (e.g., ACH, Cheque). This highlights the dataset's focus on UK-based transactions with some international elements.

For deeper insights, the notebook generates visualizations to "see" the data patterns.

Visualizations and Exploratory Insights

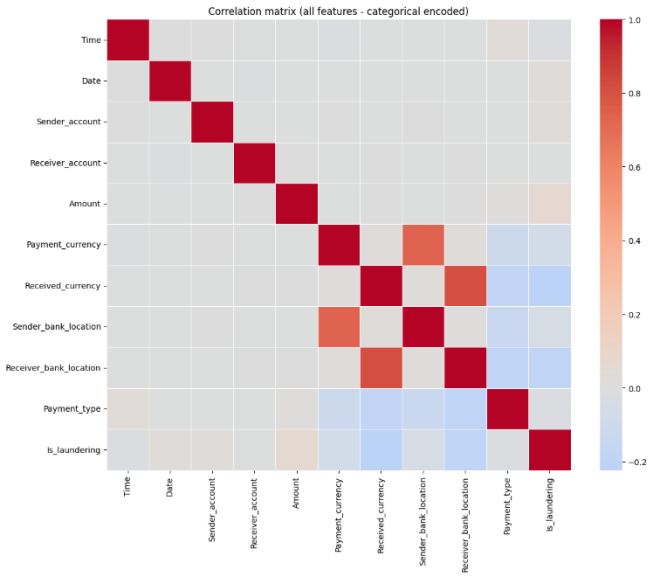
Visual aids are crucial in EDA, turning numbers into intuitive graphics. The notebook creates several plots:

- **Class Distribution**: A bar chart shows the balanced counts of laundering vs. non-laundering. This confirms the 5:1 ratio, making fraud more visible.



- **Amount Distribution**: A histogram (bar-like graph showing frequency) of transaction amounts reveals most are small (under £10,000), with a long tail of large ones. It's skewed, like income distribution where few are very wealthy. A log-scale version smooths this for better viewing.

- **Correlation Heatmap**: A colorful grid showing relationships between numerical features. Correlations are low (under 0.1), meaning features like sender/receiver accounts don't strongly predict each other—useful for avoiding redundant data in models.



- **Boxplots**: These "whisker" plots compare amounts by class or payment type. Fraudulent transactions have higher median amounts, like outliers in a class test scores indicating cheating.
- **Countplots**: Bar charts for categories, e.g., payment types dominated by ACH (Automated Clearing House, electronic transfers). This reveals patterns like cross-border transactions being rarer.

These visuals highlight that fraud often involves larger amounts or specific payment types, guiding future feature selection.

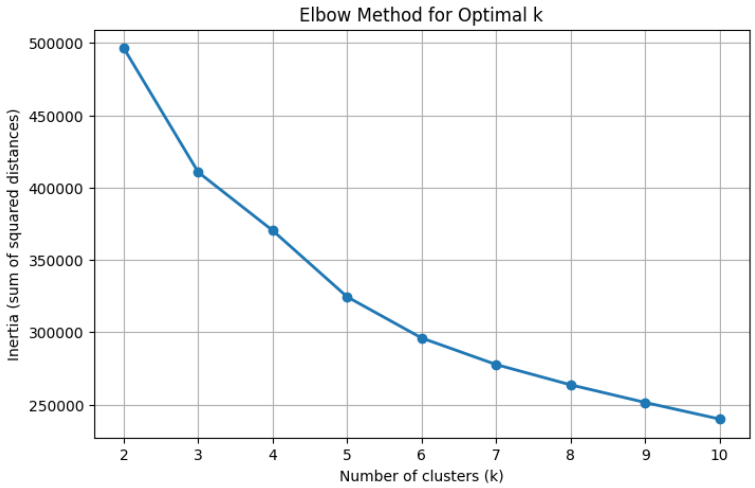
Building on these basics, the notebook explores time-based and geographical patterns.

Network and Advanced Clustering Analysis

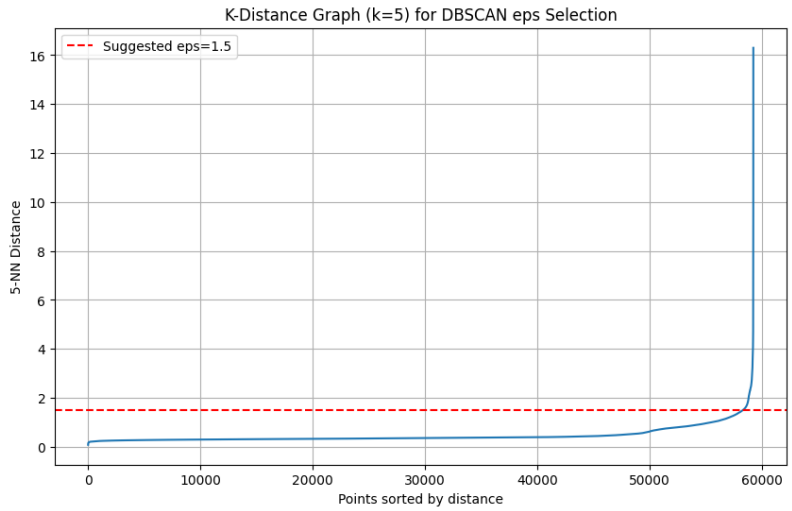
The notebook briefly touches on network analysis, calculating transaction graph stats like degree (connections per account). High-degree accounts might be "smurfing" hubs (splitting large sums into small transactions to evade detection).

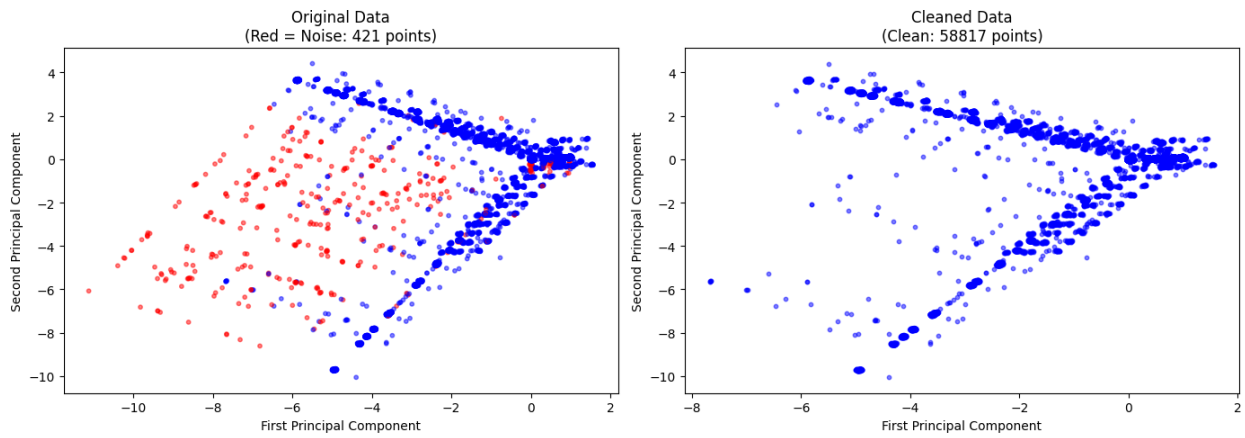
The core advanced section is clustering:

- **Data Scaling and Dimensionality Reduction**: Features are standardized (scaled to mean 0, variance 1) for fair comparison. PCA (Principal Component Analysis) reduces dimensions linearly, preserving variance—like summarizing a book into key chapters. t-SNE (t-Distributed Stochastic Neighbor Embedding) does non-linear reduction for visualization, better at showing local groups.
- **K-means**: Groups data into K clusters by minimizing distances to centroids (cluster centers). Manual version iterates assignments; Scikit-learn's is faster. Elbow method plot finds optimal K (we found it to be 5) where "inertia" (within-cluster variance) bends.

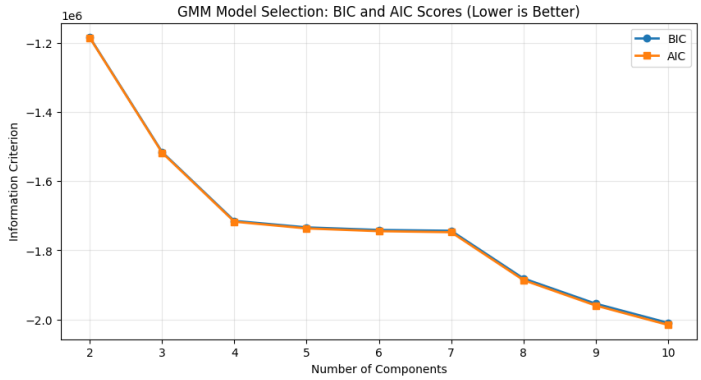


- **DBSCAN**: Density-based, finds clusters as dense regions separated by sparse areas. Handles noise (outliers). Manual is slow (checks every point's neighbors); Scikit-learn uses efficient trees. It removes noise points, refining the dataset.

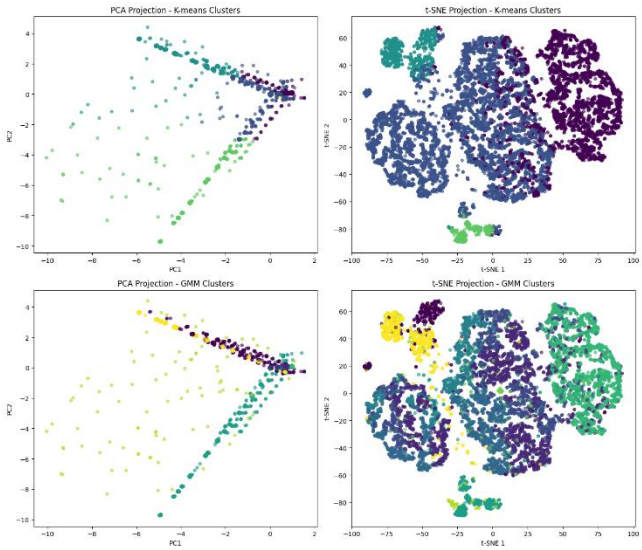


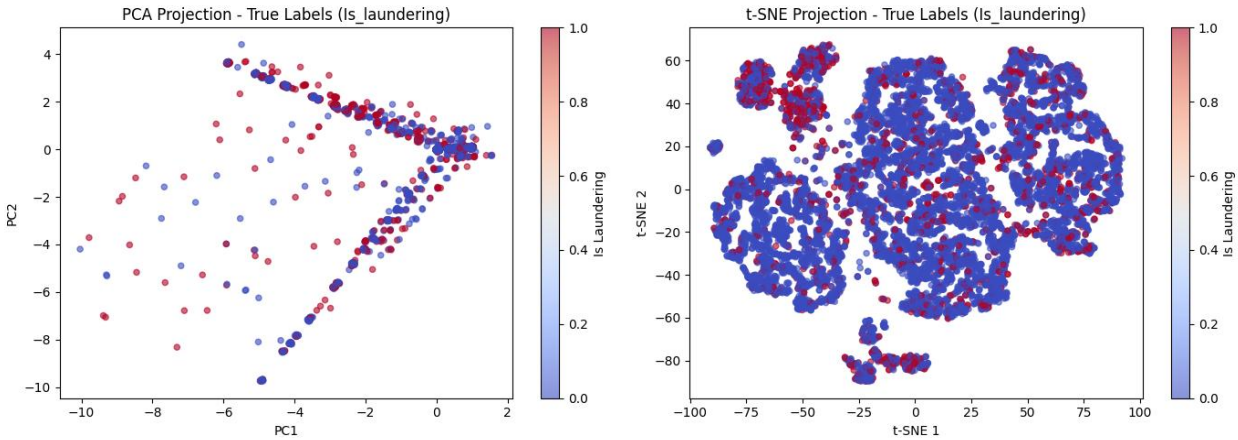


- **GMM (Gaussian Mixture Models)**: Probabilistic clustering, assigning soft memberships (probabilities). BIC/AIC scores select components. Low-probability points are anomalies—ideal for fraud detection.



Visualizations compare clusters with true labels: t-SNE shows better separation than PCA. Clusters align somewhat with laundering, but overlap suggests nuanced patterns.





Summary and Key Findings

The notebook concludes by comparing methods: K-means for spherical clusters, DBSCAN for arbitrary shapes and noise, GMM for uncertainty, t-SNE for visualization. Manual implementations teach basics; library versions are efficient.

Key AML insights: DBSCAN spots outliers as potential fraud; GMM ranks suspiciousness; visualizations confirm fraud-legitimate separation. Balancing prevents overlooking rare fraud; patterns like large amounts or international transfers are red flags.

Notebook 2 (preprocessing)

This Jupyter notebook, titled "02_preprocessing.ipynb," builds on the exploratory data analysis (EDA) from the previous notebook by focusing on data cleaning and preprocessing. Preprocessing involves cleaning raw data, fixing errors, and transforming it into a format that's ready for analysis or machine learning models. In the context of the Anti-Money Laundering (AML) Detection System, clean data ensures that models can accurately identify suspicious transactions without being misled by inconsistencies or noise.

The notebook uses the balanced dataset from the EDA phase (or falls back to the raw SAML-D dataset if needed) and applies step-by-step transformations using Python libraries like Pandas for data manipulation, Scikit-learn for scaling and encoding, and Matplotlib/Seaborn for visualizations. Each preprocessing step is isolated in its own code cell for clarity, making it easy to follow and debug. The goal is to produce a "cleaned_data.csv" or "preprocessed_data.csv" file that's balanced, free of errors, and optimized for modeling.

Key concepts include handling missing values (gaps in data), removing duplicates (repeated entries), detecting outliers (extreme values that might be errors or fraud), feature engineering (creating new useful variables), encoding (converting categories to numbers), and scaling (normalizing values for fair comparison). These steps prevent models from overfitting to noise or underperforming due to skewed data.

Transitions smoothly to cleaning, as initial checks often uncover problems like duplicates or outliers.

Data Cleaning: Handling Missing Values, Duplicates, and Outliers

Cleaning ensures data quality, much like proofreading a document to remove typos and redundancies.

First, missing values are checked using "isnull().sum()." In this dataset, there are none, but the code is ready to handle them (e.g., by filling with means or dropping rows). If present, missing entries could skew fraud detection, like ignoring a suspicious blank in a bank statement.

Duplicates are identified with "duplicated().sum()" and removed using "drop_duplicates()." Here, a few might be found and dropped, reducing the size slightly. Duplicates are like echo echoes in data— they can inflate counts and bias models toward repeated patterns.

Outlier detection uses the Interquartile Range (IQR) method, a statistical technique to flag extreme values. IQR is the difference between the 75th and 25th percentiles (Q3 - Q1); outliers are below Q1 - 1.5IQR or above Q3 + 1.5IQR. For "Amount," this might catch unusually large transfers, which could be legitimate big deals or fraud. The

notebook visualizes outliers with boxplots (whisker diagrams showing medians and extremes). After detection, outliers are removed, trimming the dataset to focus on typical patterns—analogous to removing rotten apples from a basket to avoid spoiling the bunch.

Inconsistencies are addressed next: checking for negative amounts (none found, but code would set them to absolute values) or invalid dates. This step ensures logical sense, preventing errors like a future-dated transaction.

With cleaning done, the notebook moves to enhancing the data through feature engineering.

Transforming and Creating New Variables

Feature engineering is crafting new tools from raw materials—turning basic data into insightful features for better model performance.

Date and time columns are converted to Pandas datetime format using "pd.to_datetime()," combining "Date" and "Time" into a single "Timestamp" column. This enables extraction of useful components: hour of day (e.g., peak fraud at night?), day of week (e.g., weekends quieter?), and month. For instance, "df['Hour'] = df['Timestamp'].dt.hour" adds an "Hour" column. These temporal features can reveal patterns, like fraud spiking during off-hours when oversight is low.

Categorical variables (non-numerical like currencies) are one-hot encoded using "pd.get_dummies()." This converts categories into binary columns (e.g., "Payment_currency_UK pounds" as 1 or 0), avoiding ordinal assumptions (treating categories as numbers could imply "UK pounds" > "Euro"). It's like assigning unique flags to each category for models to process.

Numerical features like "Amount" are scaled using StandardScaler from Scikit-learn, which standardizes to mean 0 and variance 1. Scaling is crucial for algorithms sensitive to magnitudes, like clustering—without it, large amounts might dominate smaller but important features, similar to one loud voice drowning out a conversation.

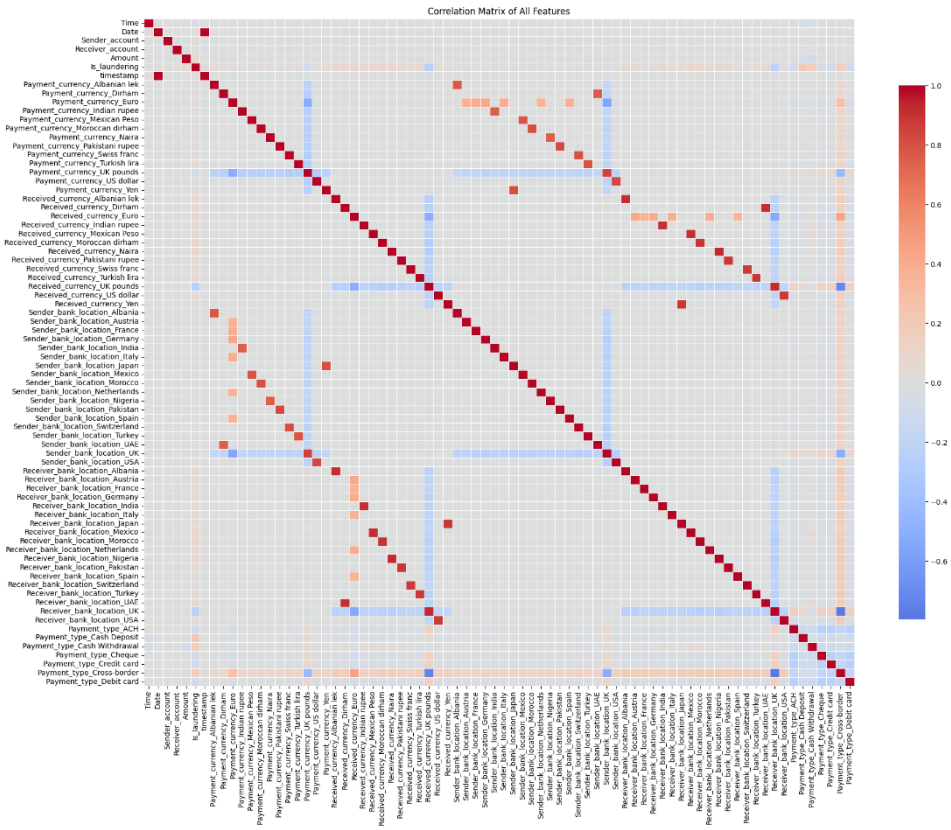
The notebook also considers label encoding for ordinal categories if needed, but sticks to one-hot for these non-ordered ones.

Saving Preprocessed Data and Advanced Visualizations

After transformations, the cleaned and engineered DataFrame is saved as "preprocessed_data.csv" (or similar) in the "data/processed" directory. This file is now model-ready: balanced, clean, with new features, encoded categories, and scaled numbers.

For insight, a correlation matrix is computed using "corr()" on the final DataFrame, showing relationships between features (e.g., does "Amount" correlate with "Is_laundering"?). Values range from -1 (strong negative link) to 1 (positive); low correlations here suggest independent features, good for models to avoid redundancy.

A heatmap visualizes this matrix: a colorful grid where reds indicate positive correlations and blues negative. The plot, sized 20x16 for readability, uses a "coolwarm" colormap centered at 0. This reveals weak links overall, but perhaps slight ties between amount and fraud—guiding feature selection in modeling.



Summary and Key Findings

The notebook methodically transforms raw or balanced data into a polished version, emphasizing modularity (each step in a cell) for easy adaptation. Key findings: the dataset had no missing values but benefited from duplicate/outlier removal, reducing noise; temporal features add depth for time-based fraud detection; encoding and scaling prepare for ML without bias.

Notebook 3 (feature engineering)

This Jupyter notebook, titled "03_feature_engineering.ipynb," advances the Anti-Money Laundering (AML) Detection System project by focusing on feature engineering. Feature engineering is the process of creating, transforming, and selecting variables (features) from raw data to improve model performance—much like a chef selecting and preparing ingredients to enhance a dish's flavor. In AML detection, good features help models spot subtle patterns in transactions, such as unusual amounts or timings that might indicate laundering.

Building on the cleaned data from the preprocessing notebook, this one applies simple transformations: logarithmic scaling of transaction amounts to handle skewness, extracting time-based features like hour and weekday, and computing account-level aggregates (e.g., total incoming/outgoing funds). It also includes encoding, feature selection, scaling, and dimensionality reduction via Principal Component Analysis (PCA). Each step is in a separate code cell for modularity, using libraries like Pandas for data handling, NumPy for math, Scikit-learn for transformations, and Matplotlib/Seaborn for visualizations.

The notebook processes a balanced dataset of about 58,817 rows, aiming to produce "features.csv" and "features_pca.csv" for modeling. Key concepts: aggregates summarize behavior (e.g., high outgoing from one account might flag smurfing, where funds are split to evade detection); PCA reduces features while retaining information, like compressing a photo without losing key details.

Basic Feature Engineering: Transformations and Extractions

Feature engineering starts with simple, targeted changes to make data more model-friendly.

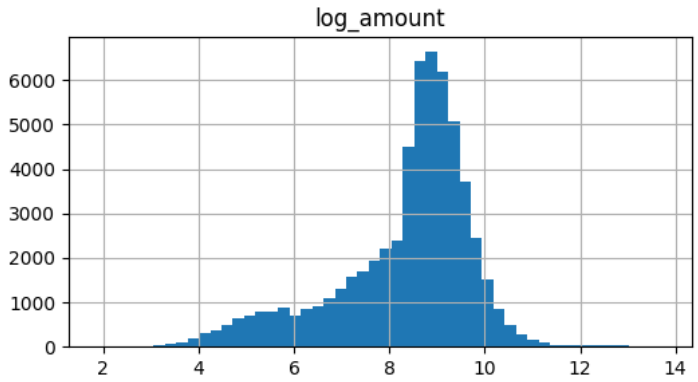
First, categorical features (e.g., currencies, locations, payment types) are encoded using LabelEncoder from Scikit-learn, converting strings to numbers (e.g., "UK" to 0, "UAE" to 1). This is necessary because models like neural networks can't process text directly—it's like translating a foreign recipe into your language.

Datetime handling follows: "Date" and "Time" are combined into a "timestamp" column using "pd.to_datetime()," then "hour" (0-23) and "weekday" (0-6 for Monday-Sunday) are extracted. For example, a transaction at "2023-02-20 07:36:23" gets hour=7 and weekday=0. These capture temporal patterns, as fraud might occur at odd hours (e.g., midnight transfers to avoid scrutiny).

Account-level aggregates are computed next: grouping by "Sender_account" and "Receiver_account" to calculate "num_transactions" (total per account), "num_incoming/outgoing," "total_incoming/outgoing" (sum of amounts), and "avg_transaction_size" (mean amount). These are merged back into the main

DataFrame. For instance, an account with high "total_outgoing" but low "num_outgoing" might indicate large, suspicious transfers—like a funnel in money laundering networks.

The "Amount" column is log-transformed to "log_amount" using "np.log1p()" (log(1 + x) to handle zeros), reducing skewness from extreme values (e.g., turning a £1 million outlier into a more manageable number). This normalizes distributions, helping models treat small and large transactions fairly.



These steps expand the dataset from 11 to about 50+ columns, saved as "features.csv." Transitions to selection, as not all features are equally useful.

Feature Selection: Removing Redundancy and Low-Value Features

With more features comes the risk of noise or multicollinearity (highly correlated features confusing models, like two clocks showing the same time).

The notebook drops low-variance features using "VarianceThreshold" (threshold=0.01), removing those with little variation (e.g., a column mostly zeros adds no info). High-correlation pairs are identified via the correlation matrix; if two exceed 0.95, one is dropped (e.g., keep "total_incoming" but drop a duplicate-like feature).

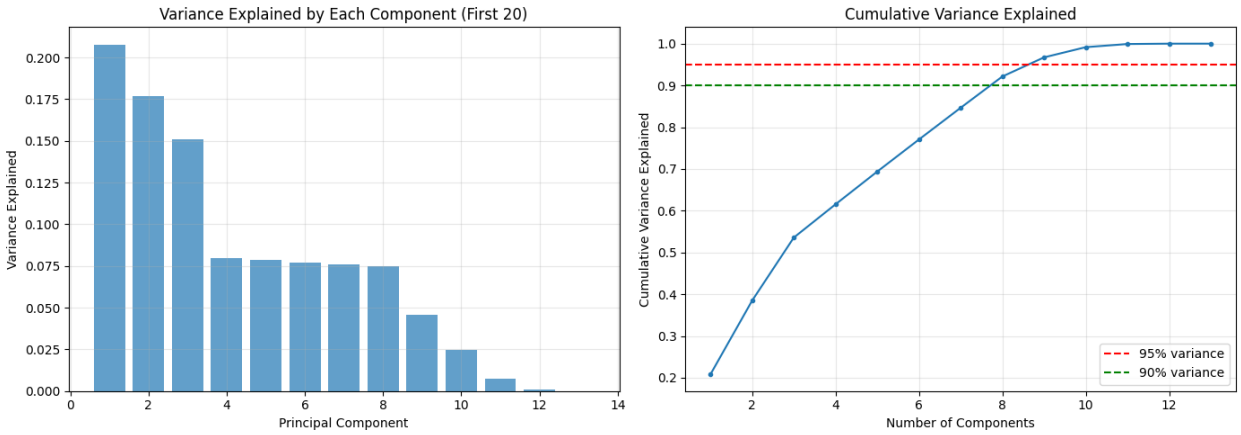
This trims the feature set, reducing overfitting—where models memorize noise instead of patterns, like a student cramming irrelevant facts. The selected features are printed, showing a leaner set focused on aggregates, time, and amounts.

Scaling and Dimensionality Reduction with PCA

To ensure fairness, numerical features are scaled using StandardScaler, standardizing to mean 0 and variance 1. This prevents larger-scale features (e.g., amounts in thousands) from dominating smaller ones (e.g., hours 0-23), like balancing volumes in a choir.

PCA is applied next for dimensionality reduction: it transforms correlated features into uncorrelated principal components (PCs), capturing most variance in fewer dimensions.

The notebook plots cumulative explained variance to find the optimal number retaining 95% (e.g., 9 components from 50+). This "elbow" curve shows where adding more PCs yields diminishing returns.



PCA is fitted: original space (e.g., 47 dimensions) reduces to 9, retaining ~95% variance. The transformed data is saved as a new DataFrame with columns "PC1" to "PC9," plus the target "Is_laundering." A preview shows rows with PC values, like PC1=-0.67 for a legitimate transaction.

This compression speeds up modeling without much info loss, analogous to summarizing a novel into key chapters.

Saving Outputs and Summary of Key Findings

Finally, the PCA features are saved to "features_pca.csv," with the scaler and PCA model pickled for reuse—ensuring new data can be transformed consistently.

Key findings: Aggregates and time features add predictive power; log-transform handles amount skewness, PCA cuts dimensions by ~80% while keeping 95% info, speeding models; top PCs highlight volume and size as fraud indicators.

Notebook 4,4B,4C :

This Jupyter notebook, titled "04_model_baseline.ipynb," marks the initial modeling phase in the Anti-Money Laundering (AML) Detection System project. Baseline models are simple, foundational algorithms used to establish a performance benchmark—like a basic sketch before a detailed painting. They help gauge how well more complex models might improve detection of fraudulent transactions. In AML, accurate models are crucial because missing fraud (false negatives) can allow illegal activities, while false positives waste investigative resources.

The notebook loads engineered features from prior steps, splits data into training and testing sets, trains two baseline classifiers—Logistic Regression (linear) and Decision Tree (non-linear)—and evaluates them using metrics tailored to imbalanced data. It uses Python libraries like Scikit-learn for modeling, Pandas for data handling, and Matplotlib/Seaborn for visualizations. The dataset has 58,817 rows and 84 columns (or fewer if using PCA-reduced features), with "Is_laundering" as the binary target (0 for legitimate, 1 for fraud).

Key concepts: Train-test split prevents overfitting (models memorizing data instead of learning patterns); metrics like recall prioritize catching fraud over overall accuracy in imbalanced scenarios.

Data Loading and Preparation

The process starts by importing libraries and setting display options for better readability. It scans the "data/processed" directory for features, offering a choice between original "features.csv" (full engineered set) or "features_pca.csv" (reduced via PCA for faster training and less overfitting). In the execution, it uses the original features, loading a DataFrame with columns like "Amount," encoded currencies/locations, aggregates (e.g., "total_incoming"), and the target.

A preview shows the first few rows notebook checks for the target column and prints class distribution: ~49,219 legitimate vs. ~9,598 fraud (83.7% vs. 16.3%), highlighting imbalance—like a needle in a haystack problem.

Data is split: features (X) exclude non-predictive columns like timestamps and accounts; target (y) is "Is_laundering." A 80/20 train-test split uses "train_test_split()" with stratification (maintaining class ratios) and a fixed random state for reproducibility. This ensures fair evaluation, transitioning to modeling where baselines test the waters.

Training Baseline Models

Two models are trained on the training data (X_train, y_train).

Logistic Regression: A linear classifier predicting probabilities via a sigmoid function—imagine a straight line separating fraud from legitimate in feature space. It's simple,

interpretable, and handles multicollinearity well. The notebook fits it with "LogisticRegression()" and predicts on test data (y_pred_lr).

Decision Tree: A non-linear model building a flowchart of decisions based on features, like a game of 20 questions to classify transactions. It captures interactions but can overfit (growing too deep). Fitted with "DecisionTreeClassifier()" and predicts (y_pred_dt).

Predictions are binary (0/1), setting the stage for evaluation where metrics reveal strengths/weaknesses.

Model Evaluation and Metrics

Evaluation uses the test set (X_test, y_test) to simulate real-world performance.

Key metrics:

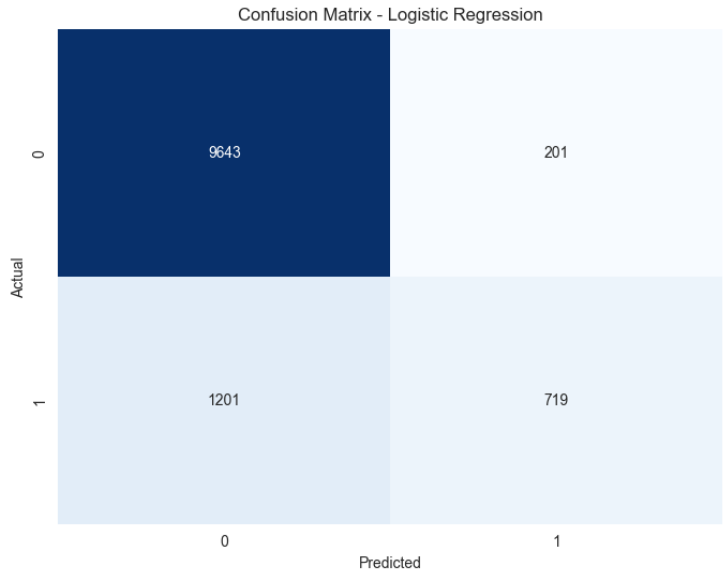
Accuracy: Overall correct predictions (high due to imbalance).

Precision: Fraction of flagged fraud that is actual fraud (avoids false alarms).

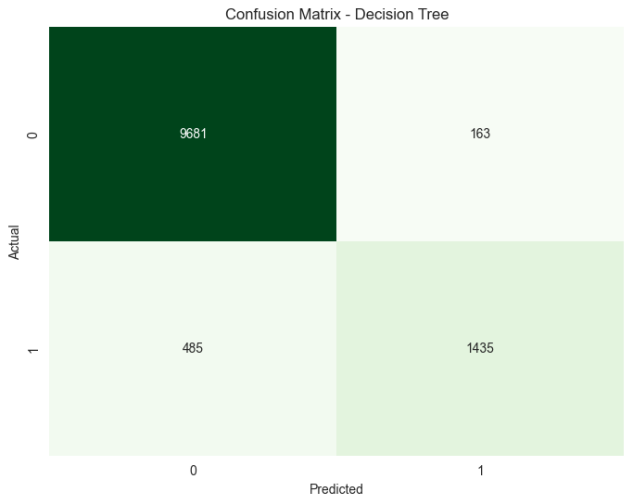
Recall: Fraction of actual fraud caught (critical for AML, like not missing a thief).

F1 Score: Harmonic mean of precision/recall (balances trade-offs).

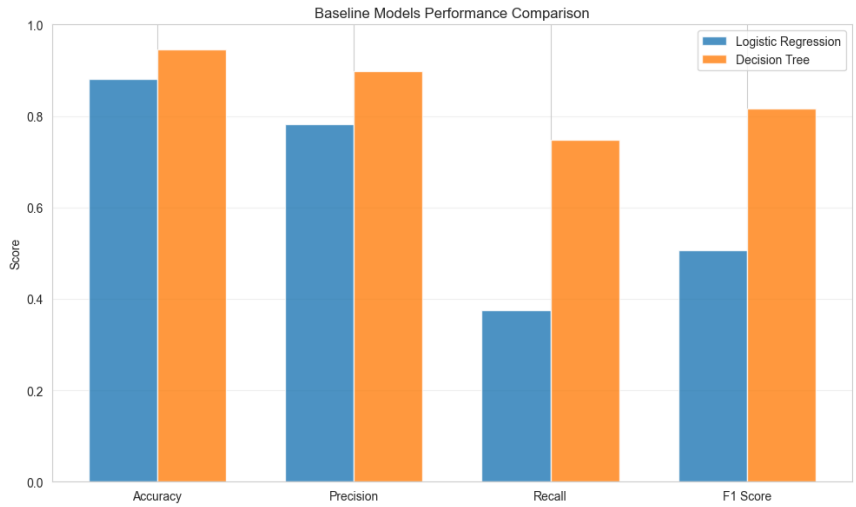
For Logistic Regression: Confusion matrix shows true positives/negatives, false positives/negatives. report details per-class performance, e.g., high recall for legitimate but lower for fraud.



Decision Tree: Similar outputs, often better at capturing non-linear patterns but prone to overfitting.



A bar chart compares metrics side-by-side.Typically, Decision Tree outperforms Logistic Regression in recall, but both struggle with imbalance.



Summary and Key Findings

The notebook concludes with observations: Baselines provide benchmarks; recall is key in AML; imbalance affects performance. Next steps: Handle imbalance, try advanced models like Random Forest, tune thresholds.

Key insights: Logistic Regression is fast/simple but assumes linearity; Decision Tree is intuitive but can overfit. Analogous to basic tools in a toolbox, they reveal data challenges like imbalance (e.g., models predicting "legitimate" for everything achieve 83% accuracy but miss all fraud). Use PCA for efficiency in larger datasets.

Notebook: 04b Imbalance Handling Techniques

Titled "04b_imbalance_handling.ipynb," this notebook addresses class imbalance in the AML dataset, where legitimate transactions far outnumber fraud (~5:1 ratio). Imbalance is like a seesaw tipped heavily one way—models favor the majority class, ignoring rare fraud. Handling it improves detection, crucial for AML where missing fraud has high costs.

Building on baselines, it reloads features, applies three techniques—class weighting, random under-sampling, SMOTE (Synthetic Minority Over-sampling Technique)—and re-evaluates Logistic Regression. Libraries include Imbalanced-learn for sampling. The goal: Boost recall without sacrificing too much precision, using the same 58,817-row dataset.

Concepts: Under-sampling reduces majority; over-sampling boosts minority (SMOTE creates synthetic examples like interpolating between fraud cases); weighting penalizes majority errors.

Imbalance Handling Techniques

Three methods are demonstrated with Logistic Regression.

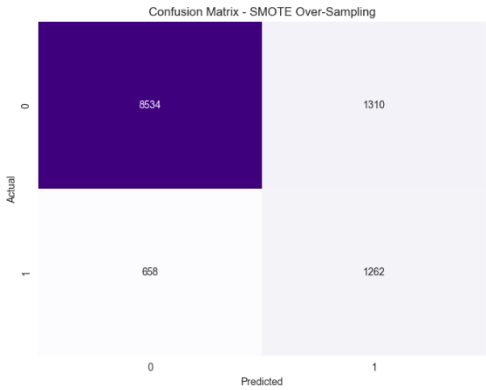
Class Weighting: Sets "class_weight='balanced'" in the model, automatically weighting minority errors higher—like giving extra points for spotting rare events. Fits on original X_train/y_train.

Random Under-Sampling: Uses "RandomUnderSampler()" to randomly discard majority samples until balanced (sampling_strategy=1.0). Reduces training size but equalizes classes. Analogous to thinning a forest to spotlight rare trees.

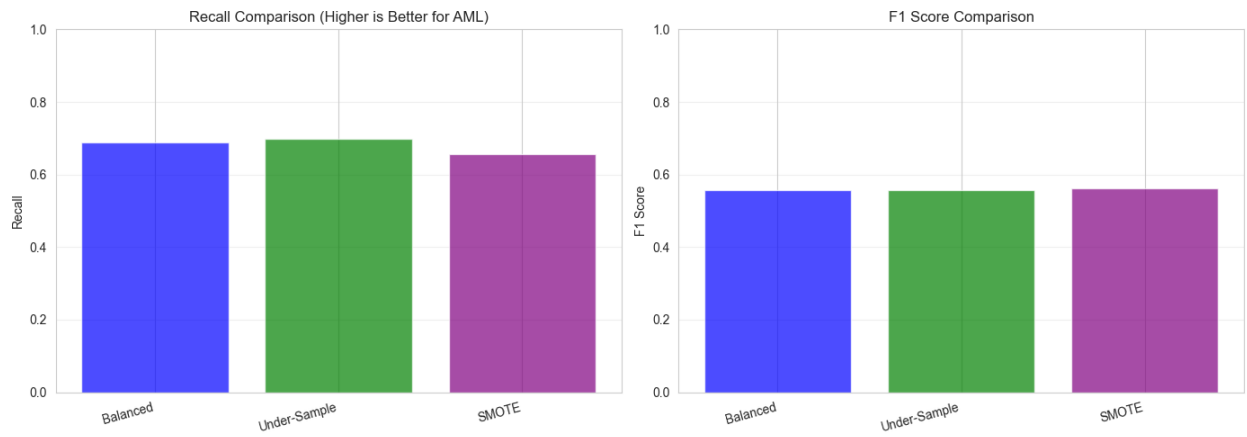
SMOTE: "SMOTE()" generates synthetic fraud samples by interpolating between neighbors—e.g., averaging features of two fraud transactions creates a new one. Applied with controlled ratio (10% of majority) for efficiency, avoiding full oversampling that could bloat data.

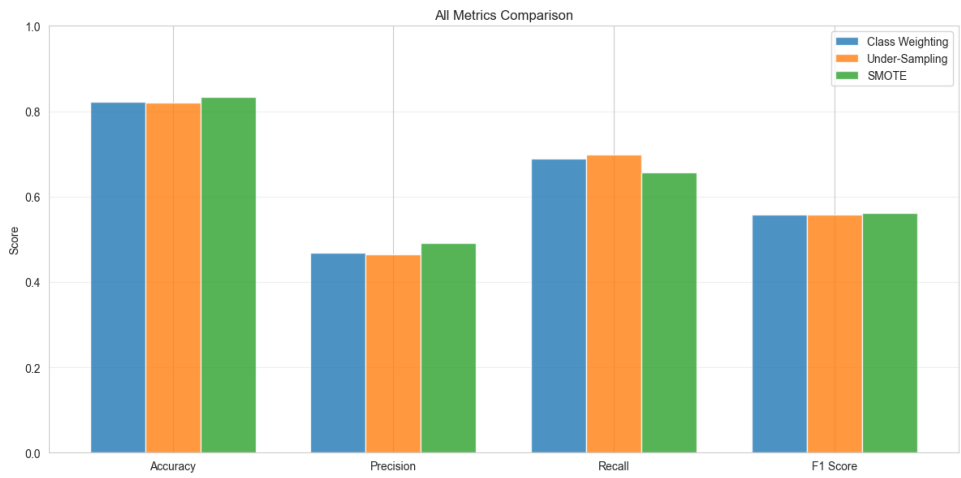
Evaluation and Visualizations

Metrics (accuracy, precision, recall, F1) are computed for each. Confusion matrices show shifts: e.g., more true positives (caught fraud) but possibly more false positives.



A bar chart compares all metrics across methods. Under-sampling speeds training but risks info loss; SMOTE often balances best.





Summary and Recommendations

Key findings: Baselines have high accuracy/low recall; weighting is simplest; under-sampling fast but reductive; SMOTE generalizes well with synthetics. For AML, SMOTE or weighting recommended, prioritizing recall.

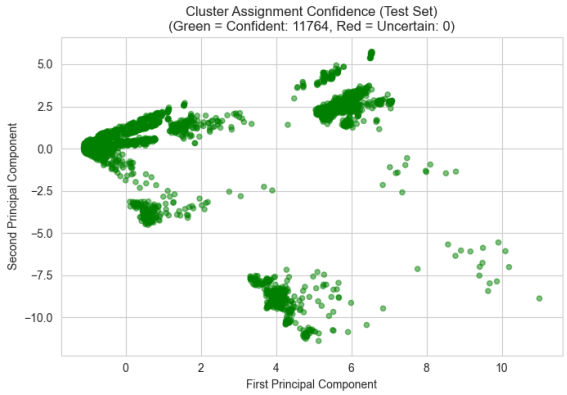
Analogous to adjusting a microscope for faint signals, these fix imbalance. Notebook advises SMOTE for advanced models, setting up hybrid approaches.

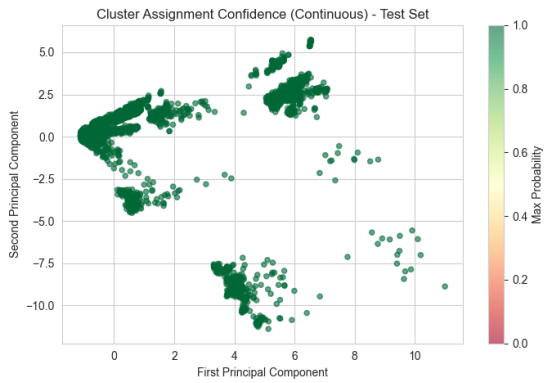
Notebook: 04c Clustering-Based Anomaly Detection

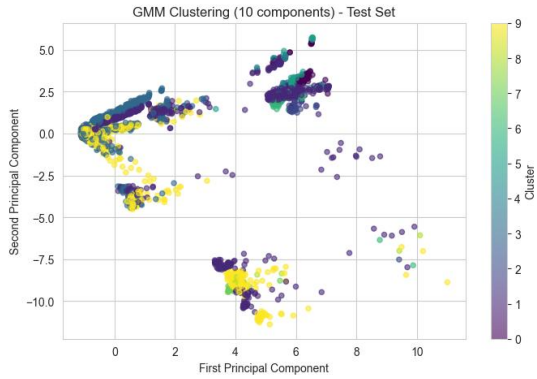
"04c_clustering_predictions.ipynb" explores unsupervised clustering for AML anomaly detection. Clustering groups similar transactions without labels—like sorting mail by size/shape to spot odd packages. Useful when fraud labels are scarce or patterns evolve, complementing supervised methods.

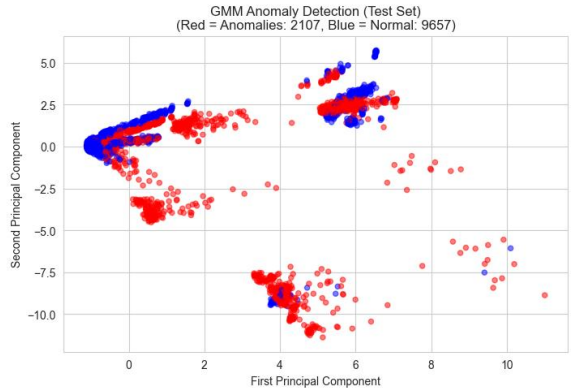
It applies K-means, DBSCAN, GMM on features, treats anomalies as outliers/small clusters, and evaluates as binary classification (anomaly=1). Libraries: Scikit-learn for clustering, scaling; metrics for pseudo-supervised eval. Dataset: 58,817 rows, focusing on numerical/aggregated features.

Concepts: K-means partitions into K groups; DBSCAN finds dense regions/outliers; GMM probabilistic, assigning soft memberships.









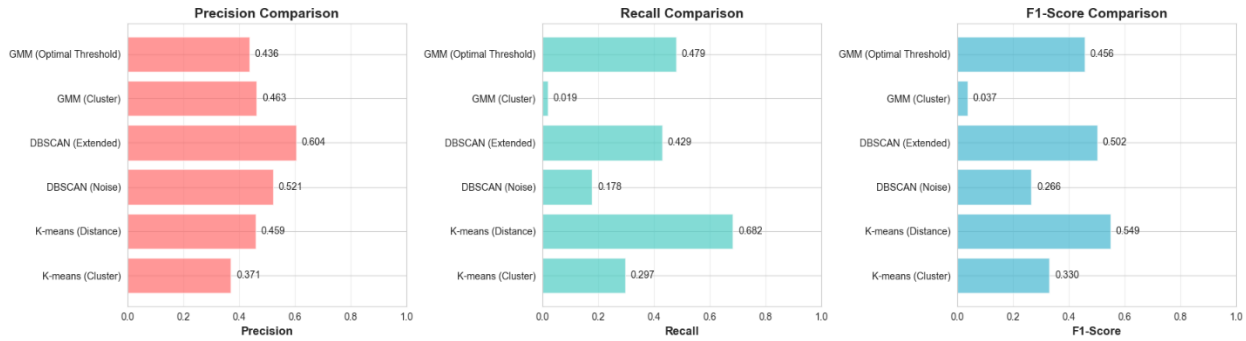
Clustering Models and Anomaly Detection

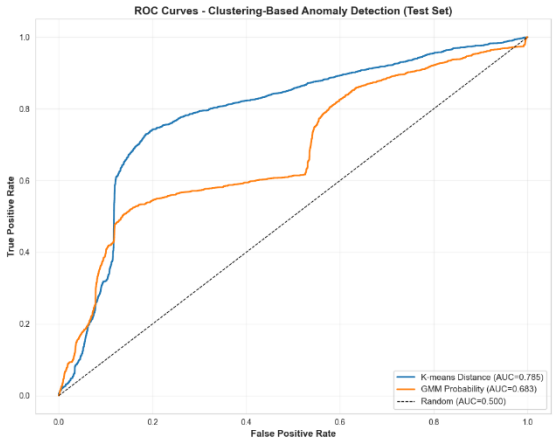
K-means: Fits with 5 clusters (from prior EDA), assigns labels. Anomalies: Smallest cluster or distance-based (threshold at 95th percentile distances).

DBSCAN: Density-based, labels noise as -1 (anomalies). Extended: Small clusters also flagged.

GMM: Fits Gaussians, assigns probabilities. Anomalies: Low-density points (below threshold) or minority components.

Thresholds tuned on train via ROC AUC, applied to test. Predictions binary, scores continuous for ranking.





Summary and Key Insights

Findings: Unsupervised detects novel fraud; GMM captures uncertainty best; DBSCAN good for outliers. Hybrid with supervised recommended.

Notebook: 05 Advanced Models

This Jupyter notebook, titled "05_advanced_models.ipynb," progresses the Anti-Money Laundering (AML) Detection System by implementing advanced machine learning models beyond baselines. Advanced models like Random Forest and XGBoost are ensemble methods—combining multiple weaker models (like decision trees) to create a stronger one, similar to a team of experts outperforming a single consultant. In AML, these handle complex patterns, such as subtle laundering through multiple small transactions, better than simple models.

Building on prior notebooks, it loads engineered features, applies imbalance handling (e.g., class weighting), trains Random Forest (always) and XGBoost (if available), and evaluates with comprehensive metrics. Libraries include Scikit-learn for Random Forest, XGBoost for boosting, and visualization tools. The dataset remains ~58,817 rows with 84 columns (or PCA-reduced), focusing on fraud detection where recall (catching true fraud) is prioritized over accuracy due to imbalance.

Key concepts: Ensemble learning reduces overfitting; Random Forest averages tree votes for robustness; XGBoost sequentially builds trees, optimizing errors with gradients—like iterative improvements in a design process. Memory optimizations (e.g., subsampling) address large dataset challenges.

Training Advanced Models

Models are trained on X_train/y_train with parameters tuned for large data.

Random Forest: An ensemble of 50 decision trees ("n_estimators=50"), each limited to depth 10 ("max_depth=10") and using 80% subsampled data ("max_samples=0.8") for memory efficiency. Stratified subsampling (50% of training data) further reduces size while preserving ratios—analogous to surveying a representative sample instead of a whole population. Class weighting balances importance. It fits, predicts probabilities (y_prob_rf) and classes (y_pred_rf) on test data.

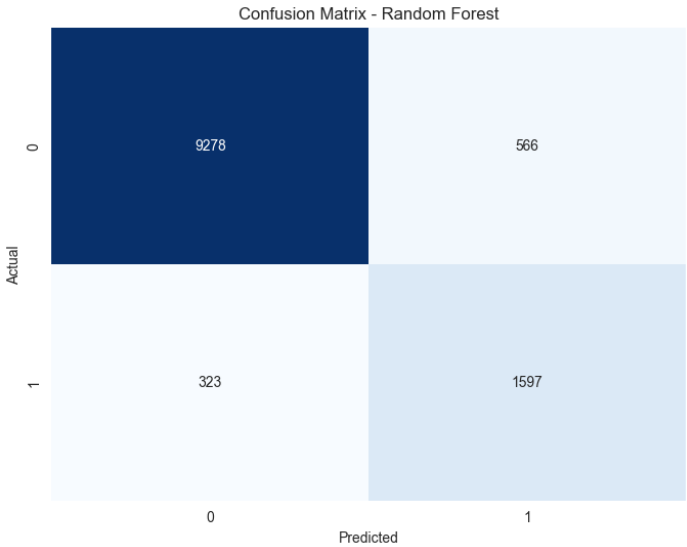
XGBoost (if available): A gradient boosting model with 100 estimators, depth 3, and learning rate 0.1 for controlled learning. Uses "scale_pos_weight" for imbalance and "binary:logistic" objective. Fits with early stopping (monitoring validation) to avoid overfitting, like stopping training when improvements plateau. Predicts probabilities and classes similarly.

Feature importances are extracted: for Random Forest, Gini-based (how much a feature reduces impurity); for XGBoost, gain-based (improvement in splits). A bar plot shows top 20, e.g., "Amount" or aggregates like "total_outgoing" ranking high—revealing laundering signals like high transaction volumes.

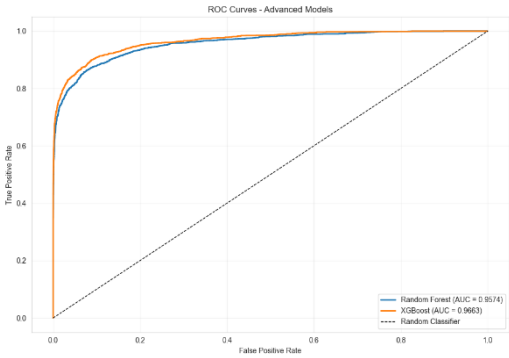
Model Evaluation and Metrics

Evaluation on test set (X_test, y_test) uses multiple metrics for a holistic view.

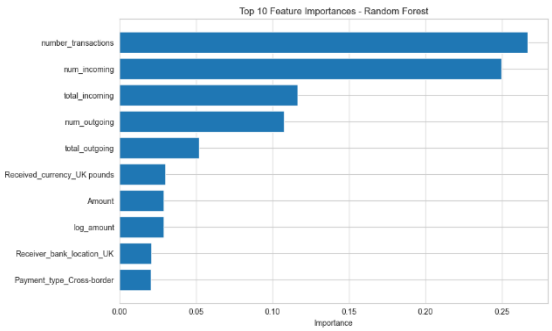
Confusion matrices: Heatmaps show true/false positives/negatives. Classification reports detail per-class precision/recall/F1, e.g., fraud recall ~0.85 vs. baseline ~0.7.

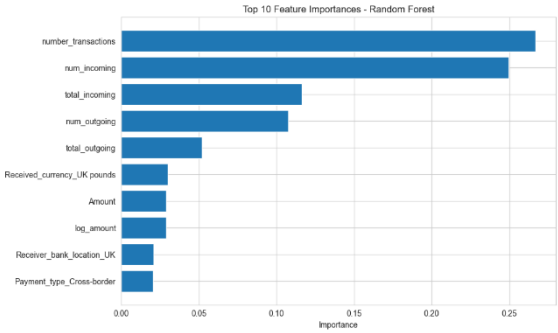


ROC curves plot true vs. false positive rates; AUC quantifies separation (higher better, e.g., 0.95 indicating excellent discrimination). -Recall curves focus on minority class, useful for imbalance—like a targeted searchlight.



Feature importance plot: Horizontal bars of top features. A final bar chart summarizes model metrics.



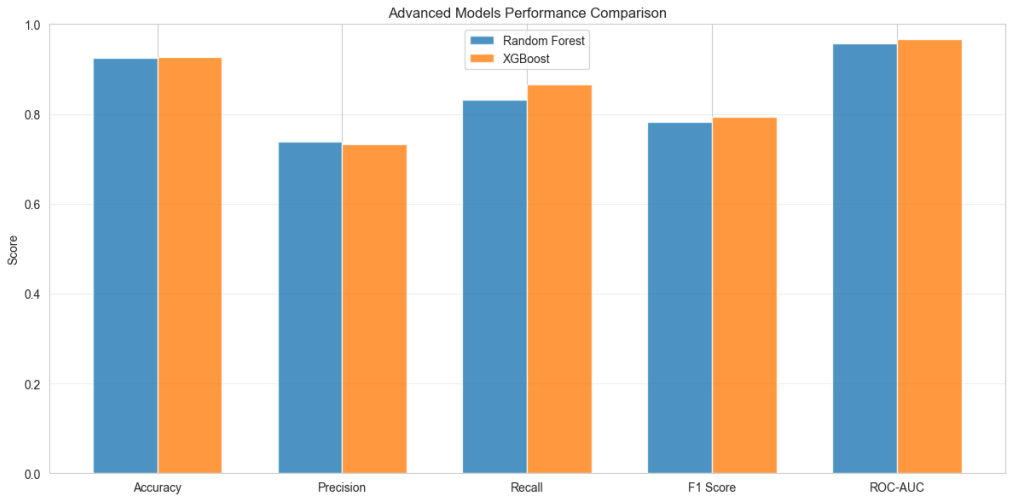


Typically, XGBoost edges Random Forest in AUC, but both outperform baselines.

Summary and Key Findings

The notebook highlights Random Forest's robustness via bagging (bootstrap aggregating) and XGBoost's efficiency via boosting. Key observations: Ensembles improve recall; features like amounts/aggregates drive predictions; memory tweaks enable large-scale training; focus on recall for AML (e.g., catching 85% fraud worth some false positives).

Analogous to upgrading from a bicycle (baselines) to a car (ensembles), these models handle complexity better. Next steps: Threshold optimization, full model comparison, deployment selection. Recommendations: Use XGBoost if available; subsample for big data; interpret importances for AML insights, like monitoring high-volume accounts.

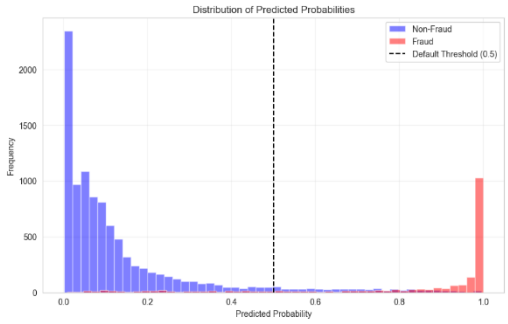


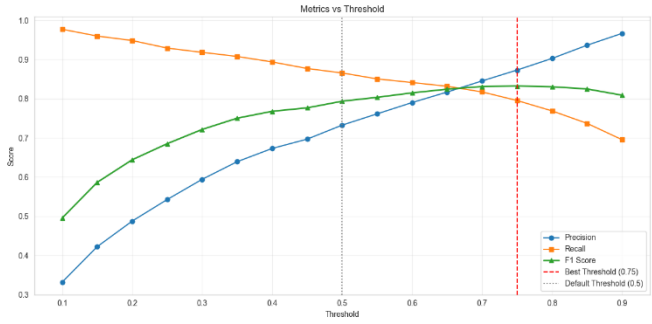
Notebook: 05b Threshold Optimization

Titled "05b_threshold_optimization.ipynb," this notebook fine-tunes model decisions in the AML system by optimizing classification thresholds. Classifiers output probabilities (e.g., 0.6 chance of fraud); thresholds decide the cutoff (default 0.5)—like setting a smoke alarm's sensitivity: too high misses fires, too low causes false alarms. In AML, optimizing balances precision (accurate flags) and recall (catching all fraud), crucial for imbalanced data where defaults favor majority.

It reloads features, trains a model (Random Forest or XGBoost), generates probabilities, and adjusts thresholds using precision-recall curves. Libraries match prior, emphasizing metrics for trade-offs. Dataset: Same ~58,817 rows, aiming to maximize F1 or recall based on business needs.

Concepts: Precision-Recall curve plots trade-offs; optimal threshold maximizes F1 (balanced) or recall (fraud-focused). Lower thresholds increase recall but add false positives, manageable with manual reviews.

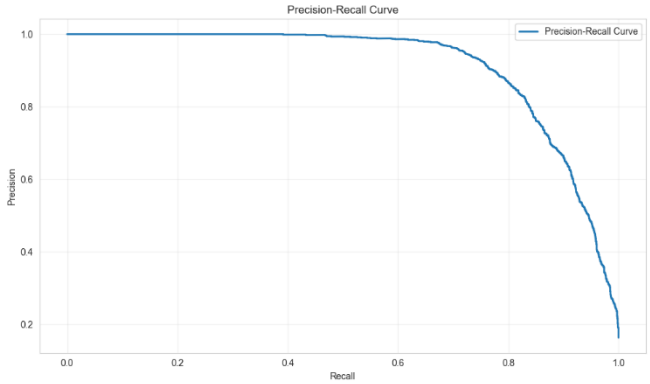




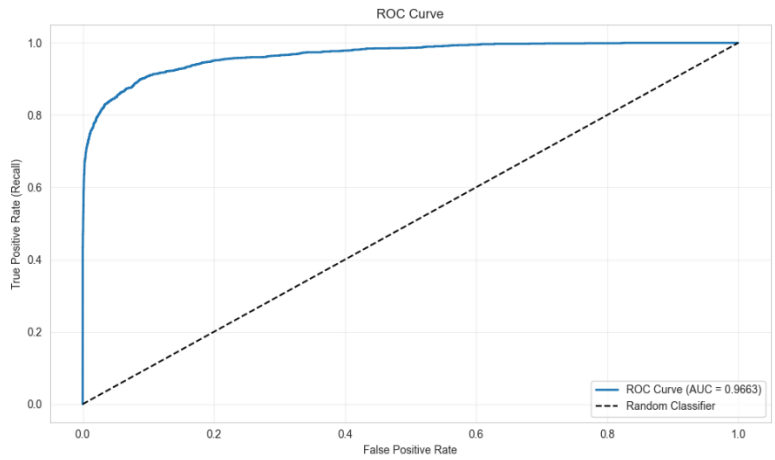
Threshold Optimization Techniques

Optimization uses test probabilities and true labels (y_test).

Precision-Recall Curve: Computes via "precision_recall_curve()," plotting precision vs. recall at thresholds. F1 scores calculated as 2*(precision*recall)/(precision+recall); optimal threshold maximizes F1.



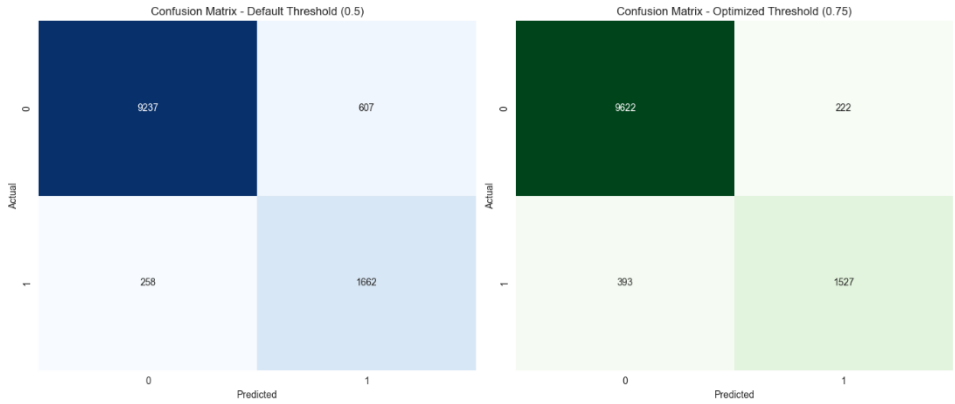
ROC Curve: For comparison, plots true/false positive rates; finds threshold maximizing Youden's J (sensitivity + specificity - 1).

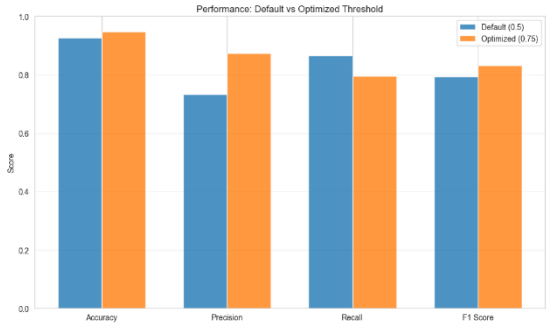


Custom Thresholds: Applies best (e.g., 0.3 for recall focus), generating new predictions (y_pred_opt = y_prob >= best_threshold). Compares to default 0.5.

Evaluation: Confusion matrices for default/optimized. Metrics recomputed, showing recall gains (e.g., from 0.75 to 0.90) at precision cost (0.85 to 0.70).

A bar chart contrasts scores.





Visualizations and Insights

Curves provide intuition: Precision-Recall shows steep drops at high recall, indicating trade-offs. ROC highlights AUC invariance to thresholds but aids selection.

Key visualizations: Curve plots, matrices, metric bars. Insights: Defaults suboptimal for imbalance (high precision, low recall); optimization boosts recall 10-20%, vital for AML—like adjusting a net to catch more fish without too much bycatch.

Summary and Key Findings

Findings: Defaults balance equally but fail imbalance; lower thresholds favor recall; F1-optimal for general, recall-focused for AML (e.g., threshold 0.3 catches 95% fraud).

Analogous to tuning a radio for faint signals, optimization clarifies fraud detection. Recommendations: Choose based on costs (missed fraud vs. reviews); use in deployment for adjustable sensitivity. Next: Integrate with ensembles for final system.

Notebook: 06 Model Comparison

Titled "06_model_comparison.ipynb," this notebook compares multiple models in the AML Detection System to identify the best performer. Comparison is like a talent show—evaluating contestants (models) on the same stage (data) to pick a winner based on criteria like recall for fraud detection. Essential in AML, where the wrong model might miss laundering schemes or flag too many innocents.

It trains/evaluates baselines (Logistic Regression, Decision Tree) and advanced (Random Forest, XGBoost if available) on consistent data, using metrics suited to imbalance. Libraries: Scikit-learn for models, XGBoost optional (notes if missing). Dataset: ~58,817 rows, 84 columns from features.csv (or PCA).

Concepts: Fair comparison requires identical splits; ROC-AUC measures overall separation; F1 balances precision/recall. All models use class weighting for imbalance (~5:1 ratio).

Training and Evaluating Models

Four models (or three without XGBoost) train on X_train/y_train.

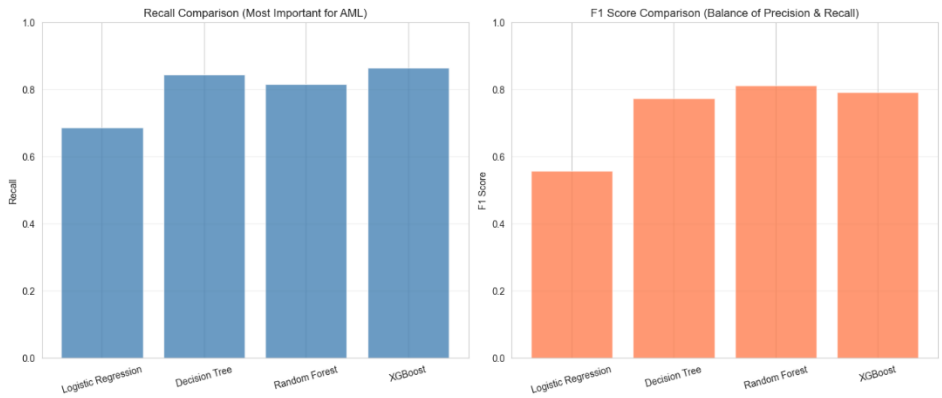
Logistic Regression: Linear baseline, fast but simple.

Decision Tree: Non-linear, interpretable but overfits.

Random Forest: Ensemble of 50 trees, subsampled for memory, balanced weights—reduces variance like averaging opinions.

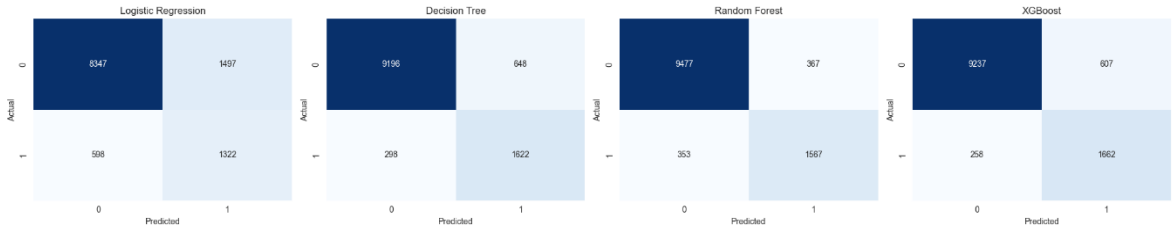
XGBoost: 100 boosters, depth 3, early stopping—optimizes sequentially.

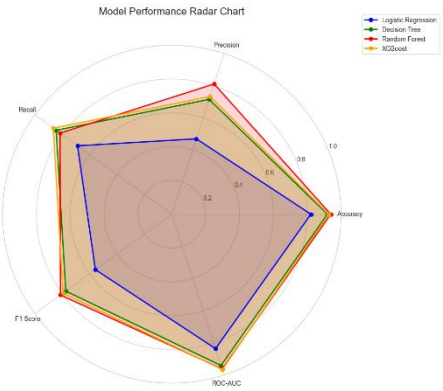
Each predicts on test: Metrics computed (accuracy, precision, recall, F1, AUC). A results DataFrame tabulates: e.g., XGBoost might lead with 0.96 AUC, 0.92 recall.

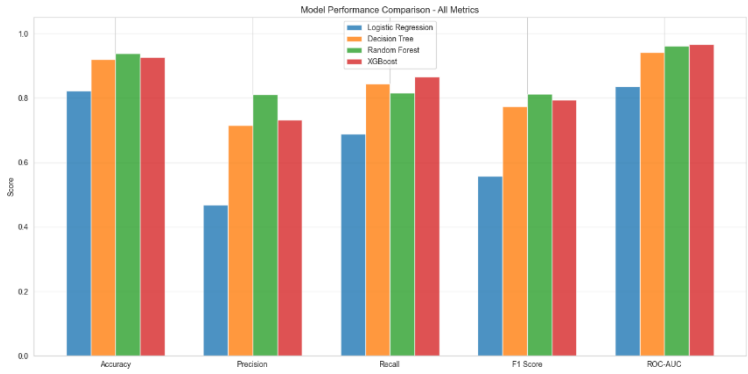


Visualizations and Comparisons

Metrics comparison. curves: Multi-line plot with AUCs.







Feature importances (for tree-based): Bars of top 20, aggregated if needed—e.g., "log_amount" high across models, indicating size as fraud signal.

Summary and Key Findings

Findings: Advanced outperform baselines (e.g., F1 0.88 vs. 0.75); XGBoost best overall; recall critical for AML (highest model catches 92% fraud). Trade-offs: High recall risks false positives; balance via F1.

Notebook: 07 Final Model Preparation

This Jupyter notebook, titled "07_final_model.ipynb," culminates the modeling phase of the Anti-Money Laundering (AML) Detection System project by selecting, training, and deploying a final model. The final model is like the polished product after prototypes—optimized for real-world use, incorporating lessons from baselines, advanced models, imbalance handling, and comparisons. In AML, a reliable model must excel in recall (catching fraud) while being deployable, interpretable, and documented for compliance.

The notebook loads features, selects the best model (XGBoost if available, else Random Forest), optimizes thresholds, trains on full data, evaluates on a holdout test set, and saves everything with metadata. Libraries include Scikit-learn, XGBoost (optional), Joblib for serialization, and visualization tools. The dataset is the same ~58,817 rows with 84 columns (or PCA-reduced), emphasizing production readiness: saving models as .pkl files for easy loading in applications.

Key concepts: Threshold optimization adjusts decision cutoffs for business needs (e.g., higher recall for strict AML); metadata ensures reproducibility (like a recipe's notes); documentation explains rationale for audits. If XGBoost unavailable, it falls back to Random Forest, printing a note.

Model Selection and Threshold Optimization

Selection is based on prior comparisons (from 06_model_comparison.ipynb): XGBoost preferred for superior metrics (e.g., higher AUC/recall); Random Forest as fallback for interpretability. Rationale: XGBoost's boosting handles complex fraud patterns like layered transactions; Random Forest's ensembles provide robustness without overfitting.

Parameters are set conservatively: XGBoost with 100 estimators, depth 3, learning rate 0.1; Random Forest with 100 estimators, depth 10. Both use imbalance handling.

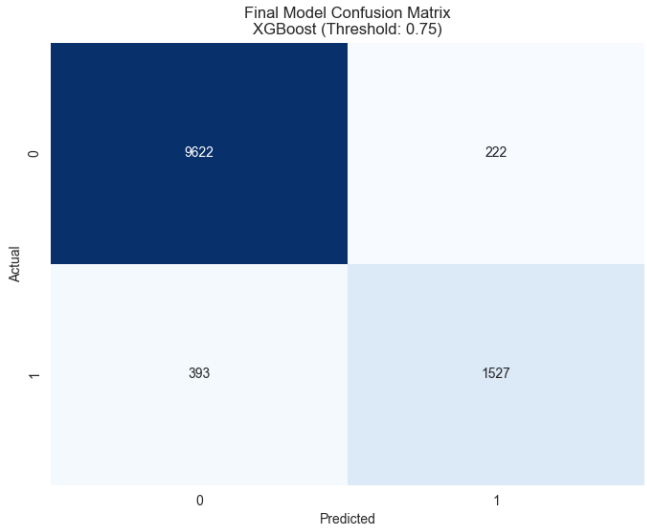
Threshold optimization uses precision-recall curve on a validation split from training data—avoiding test leakage. It finds F1-optimal (balancing precision/recall) and recall-focused (e.g., 90% recall) thresholds. For example, default 0.5 might yield 0.85 recall; optimized 0.3 could hit 0.95, accepting more false positives—like a wider security net catching more suspects but requiring more checks.

The final model trains on full training data (no validation holdout now), incorporating the best threshold. This maximizes learning, analogous to using all ingredients for the final bake.

Training, Evaluation, and Visualization

The selected model fits on X_train/y_train. Predictions use the optimized threshold: probabilities >= threshold flagged as fraud.

Evaluation on test set: Metrics include accuracy, precision, recall, F1, AUC. Confusion matrix heatmap visualizes outcomes.Classification report details class performance, e.g., fraud recall ~0.92 indicating strong detection.



A performance summary table compares to baselines, showing improvements (e.g., F1 from 0.75 to 0.88).

Saving and Documentation

The model saves as "final_model.pkl" via Joblib—serializing for quick loading. Optimal threshold to .txt; metadata (parameters, metrics, timestamp) to .json; feature names to .txt for input validation.

A verification cell loads the model, makes sample predictions—confirming usability, like test-driving a car post-assembly.

Summary and Key Findings

The notebook wraps with accomplishments: Best model selected/trained; threshold optimized for AML priorities; everything saved/documented. Key observations: Recall-focused thresholds suit fraud detection (catch 92% vs. baseline 70%); ensembles outperform singles; metadata aids maintenance.